Dynamic Batching & Sequence Packing

Fill every micro-batch with real tokens, attention stays per sample

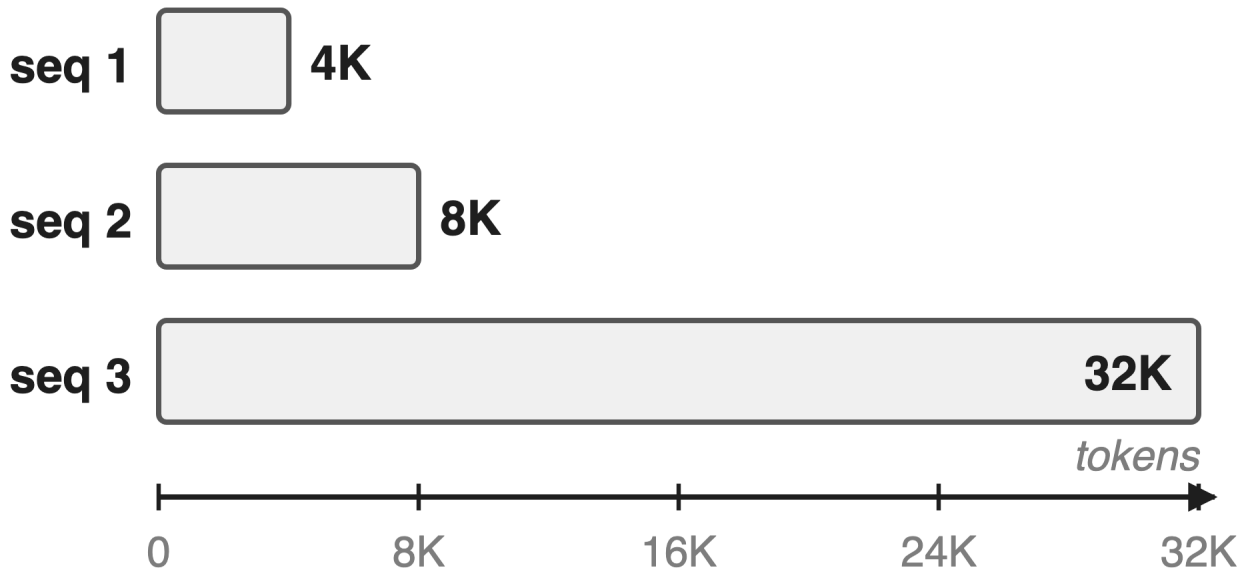
VeOmni (ByteDance Seed) · Megatron-Core THD · GLM-5

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Memory & Offload Scheduling · arxiv.org/abs/2508.02317

Real batches mix 4K and 32K samples

- Multimodal: image and video token counts vary
- GLM-5 mid-training: context grows **4K → 200K**
- RL traces: hundreds to tens of thousands of tokens



Padding wastes most of the batch

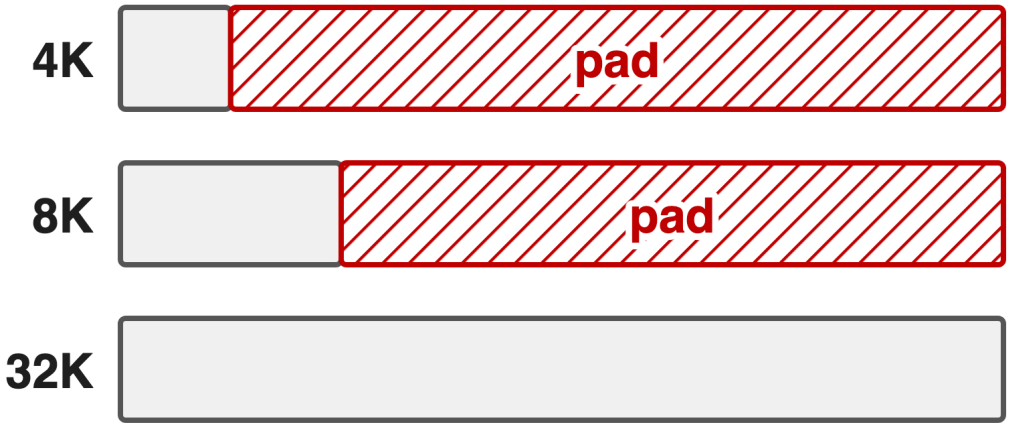
~60%

of compute and memory wasted padding 4K and 8K samples to 32K

Megatron-Core MoE technical report, arXiv 2603.07685, §6.4 (p.59)

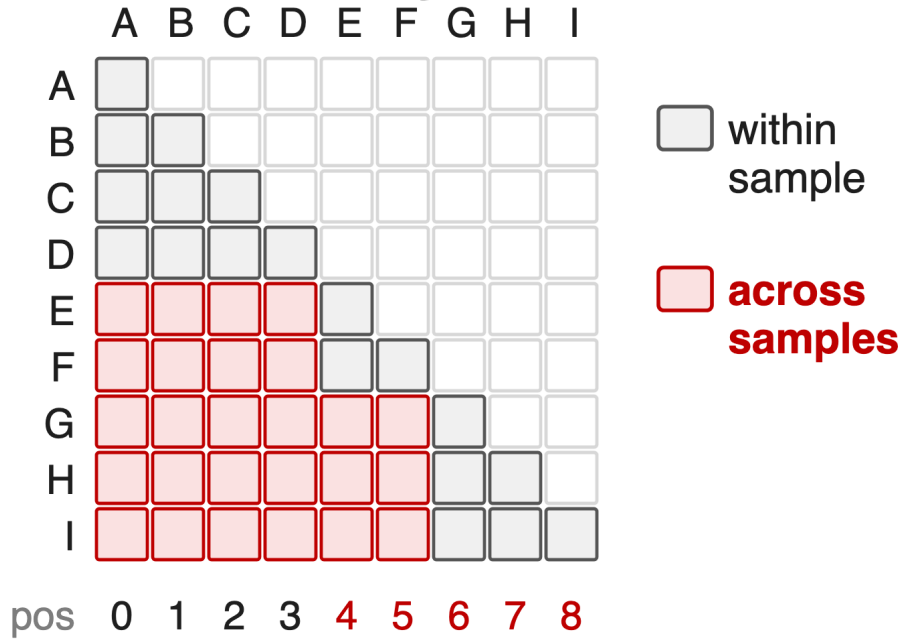
Padding wastes FLOPs; concat leaks attention

(a) Pad to the longest



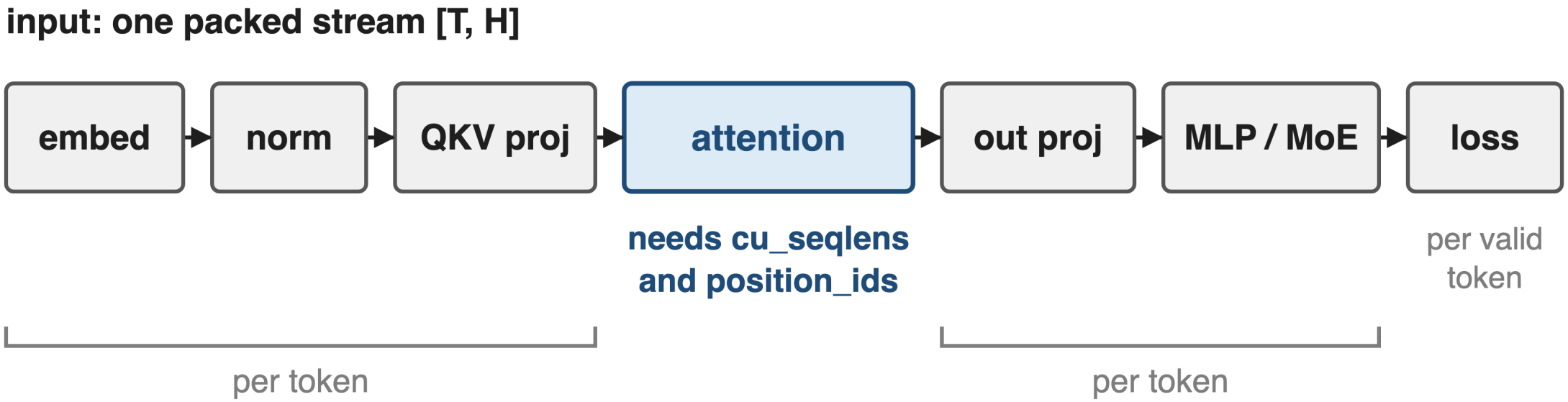
rectangle [3, 32K]

(b) Concatenate naively



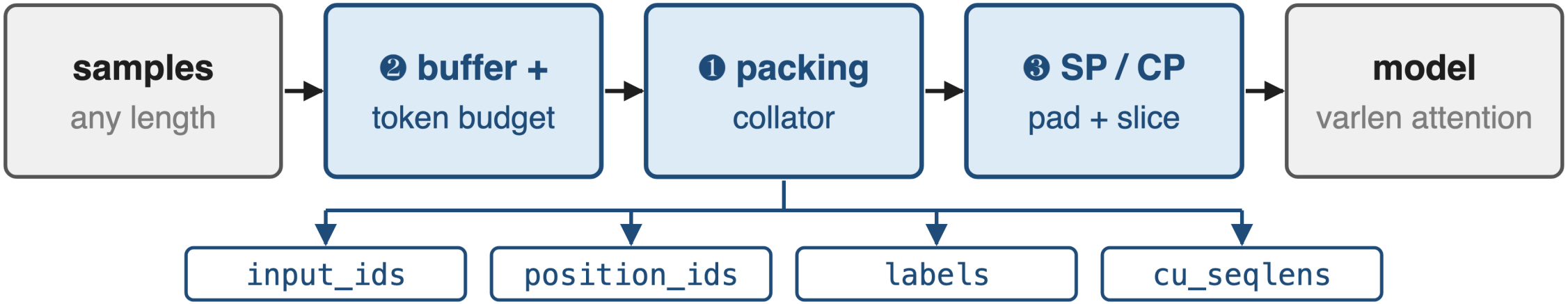
Pack without padding, but keep every sample's attention and positions its own

Insight: only attention needs sample boundaries



Flatten to one token stream; hand attention the offsets as cu_seqlens

Pack, budget, then shard the token stream

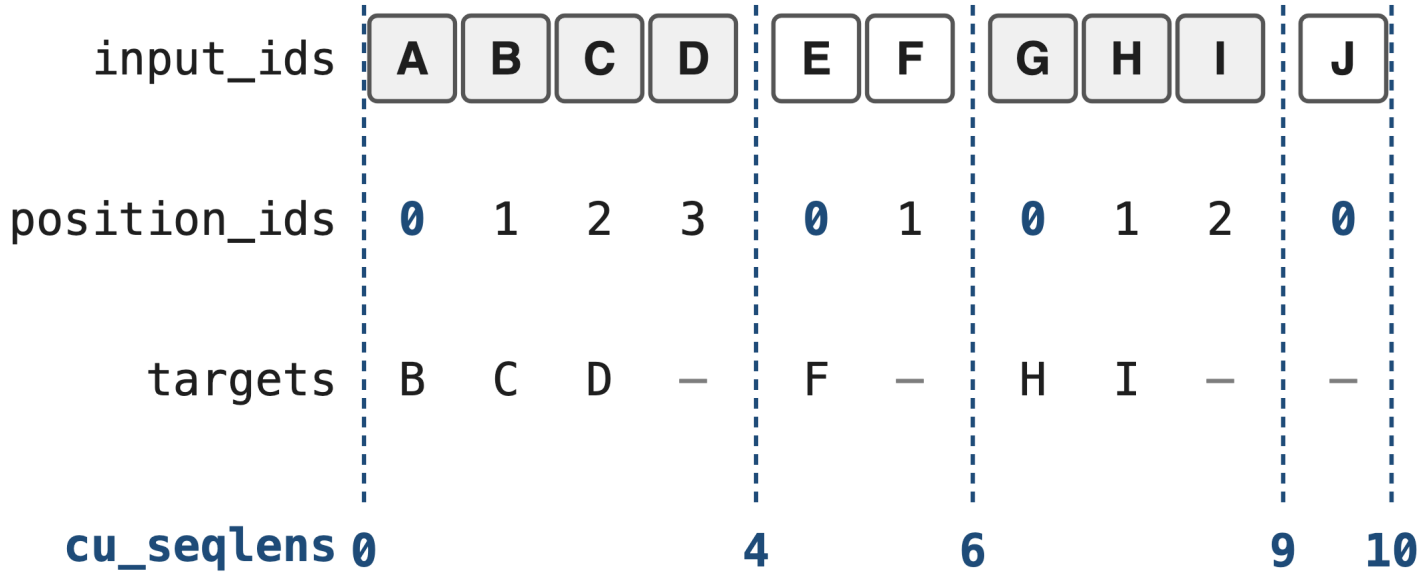


one micro-batch = one token stream

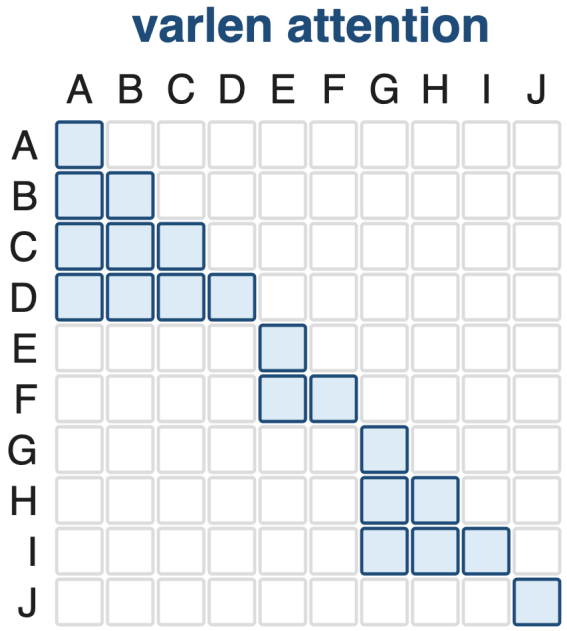
Megatron-Core: THD tensors [T, heads, dim] + cu_seqlens

VeOmni: DynamicBatchSizeDataLoader + PackingCollator + SequenceParallelCollator; Megatron-Core: THD + PackedSeqParams

❶ cu_seqlens turn one stream into four samples



targets "–" = IGNORE_INDEX



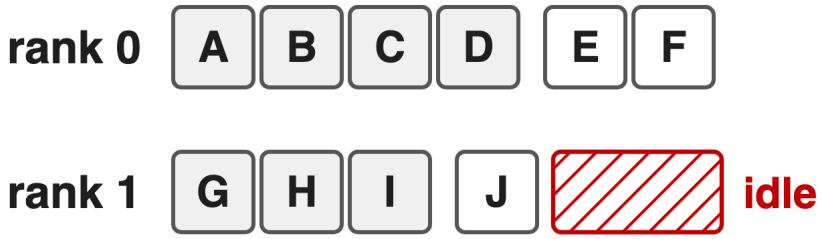
block-diagonal causal

FlashAttention varlen applies this mask from cu_seqlens; nothing is padded or materialized

❷ Fill each batch to a token budget

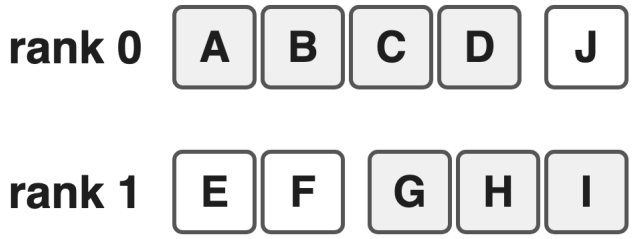


(a) 2 samples per rank



6 vs 4 tokens: rank 1 waits

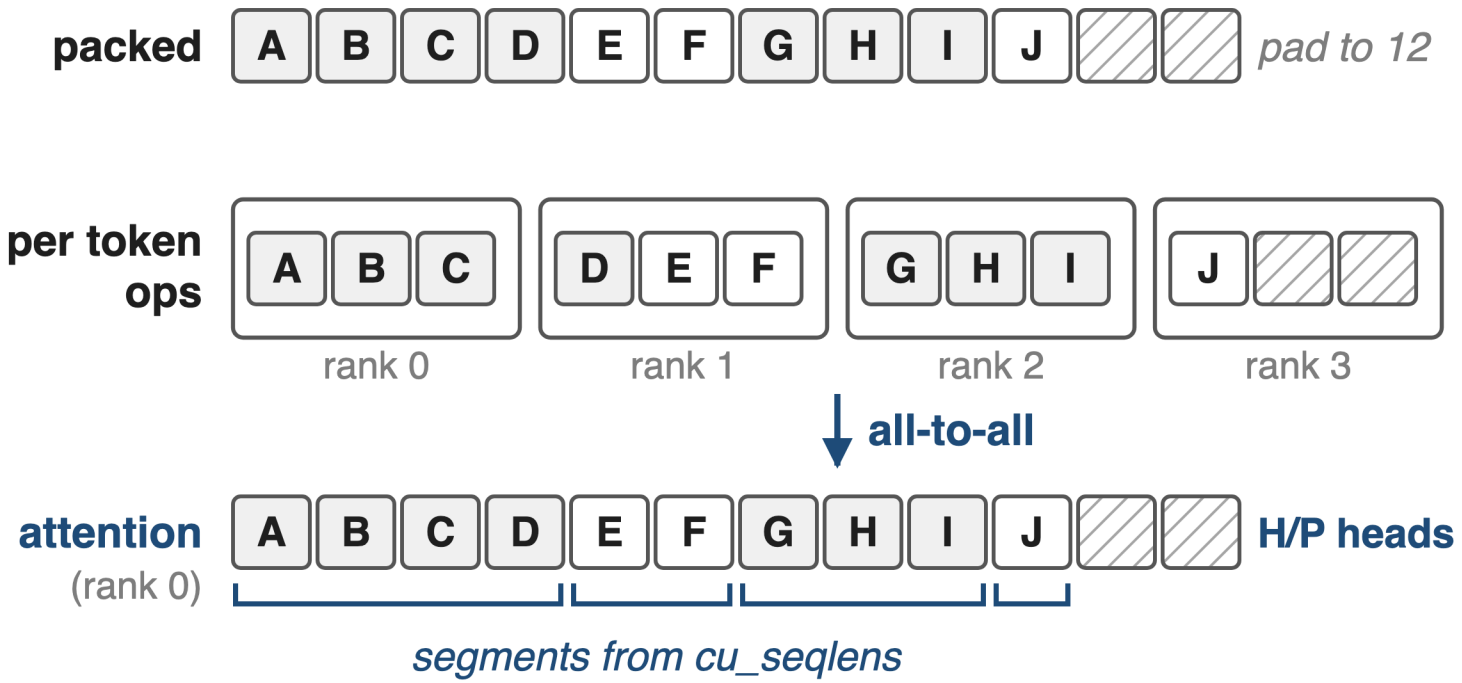
(b) token budget = 5



5 and 5 tokens: no wait

VeOmni dyn_bsz: buffer of ≥ 500 samples, first-fit to the budget; count all tokens or only loss tokens (physical cap 1.5×)

❸ A packed stream splits cleanly across SP ranks



Megatron-Core

THD [T, heads, dim]

ring CP over T

padded + unpadded

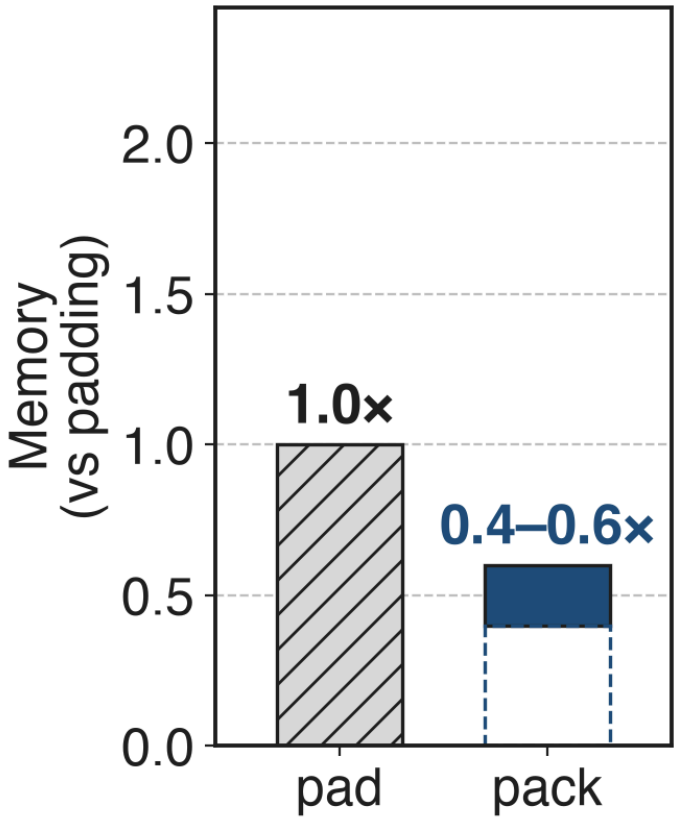
cu_seqlens

Dynamic-CP: deck 68

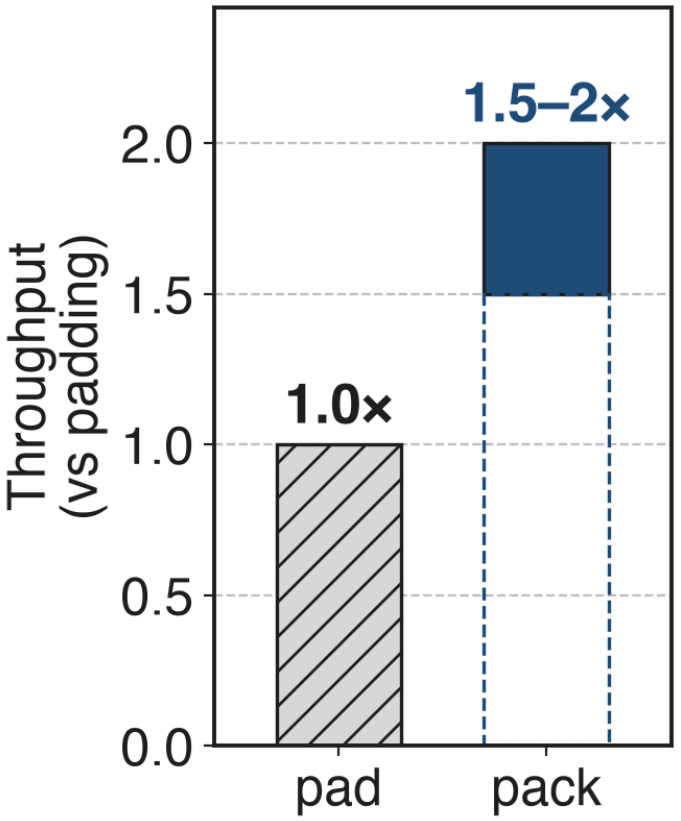
VeOmni pads only the stream's tail, to a multiple of the SP size

Packing: 1.5–2× throughput, 40–60% less memory

- Megatron-Core, RL with thinking models
- Lengths: hundreds to tens of thousands of tokens
- Gain grows beyond 32K context



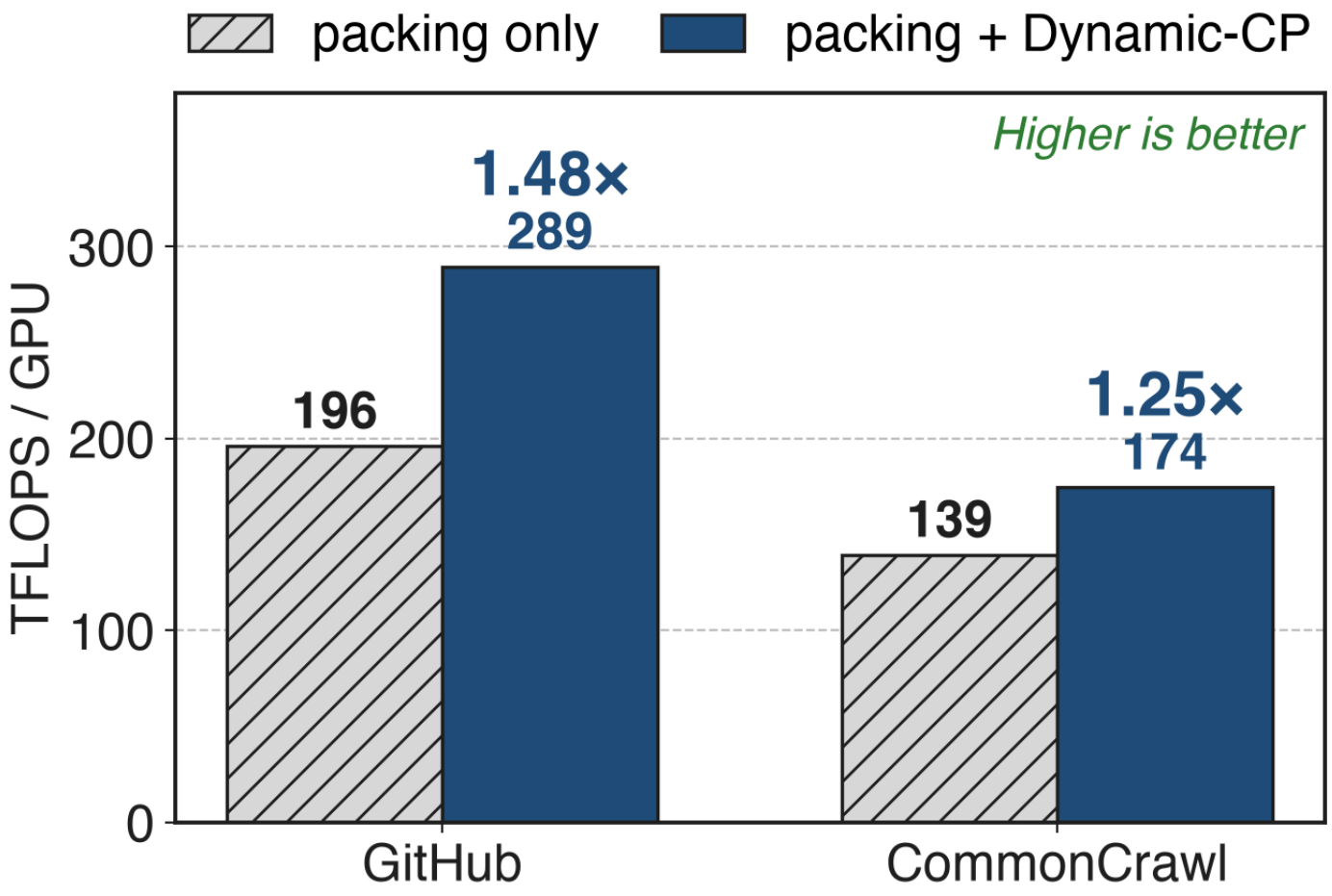
Lower is better



Higher is better

Balancing the packs adds up to 1.48×

- Llama 13B, PP 8, CP 8, batch 2048
- Equal tokens still mean unequal attention FLOPs
- **> 35%** end to end on thousands of GPUs



The cost: kernels, static shapes, uneven FLOPs

- **Varlen kernels** required: FlashAttention, TE fused attention
- **Static-shape features** (compile, CUDA graphs) need `pad_to_length`
- Equal tokens ≠ equal FLOPs without a scheduler
- Loss averaged per valid token
- VeOmni batch buffer not yet checkpointed

Packing removes padding; balancing the work is a separate problem

Summary

- Padding a 4K/8K/32K batch: **~60%** of compute and memory wasted
- **Insight**: only attention needs boundaries, so pack and pass cu_seqlens
- **1.5–2×** throughput, **40–60%** less memory; Dynamic-CP **1.25–1.48×** more
- Parent: deck 28 (VeOmni); related 68 (CP / Dynamic-CP)

https://arxiv.org/abs/2508.02317